

Virtual-Swapped TreePIR: 面向任意固定 Merkle 树的直接祖先染色与批量私有证明 检索

论文草稿

2026 年 4 月 3 日

摘要

本文研究固定 Merkle 树上的私有证明检索问题。TreePIR 针对 perfect binary tree 提出了基于祖先染色的高效检索框架，但其理论与算法均强依赖 perfect tree 结构。本文保留 swapped tree 的逻辑视角，使 Merkle proof 在查询语义上仍表现为一条自根向下的路径；与此同时，不显式物化整棵 perfect/swapped tree，也不将默认空节点存入子数据库，而仅对真实 proof-bearing node 进行直接染色与索引。基于这一 virtual-swapped 模型，本文提出一种适用于任意固定二叉 Merkle 树的祖先染色框架，并给出基于叶区间的子库索引方法，使客户端能够通过一次包含 h 个 PIR 子查询的 batch-PIR 请求恢复长度为 h 的证明，其中不存在真实节点的颜色位置由 dummy 查询补齐。进一步地，本文实现并比较了两种染色目标：以加权负载平衡为主的 **weighted** 策略，以及以子库大小平衡为主的 **count_balanced** 策略。随机树实验表明，两种策略均能稳定保持祖先异色性质；其中 **count_balanced** 能显著降低子库大小差值，而 **weighted** 在查询负载均衡方面更具优势。这些结果说明，在任意固定 Merkle 树上，存储规模均衡与查询负载均衡之间存在明确的权衡关系。

1 引言

Merkle tree 广泛应用于区块链状态承诺、证书透明性、数据库一致性证明等场景。对于一个目标叶，客户端通常需要获得一条长度为树高 h 的 membership proof 以完成验证。若客户端直接向服务器请求对应证明，则服务器可以由此推断客户端正在验证哪个目标对象，从而泄露隐私。因此，如何在暴露目标叶位置的前提下高效检索 Merkle proof，是一个自然且重要的问题。

TreePIR 通过对 perfect binary tree 进行祖先染色，将一条证明路径上的节点划分到不同颜色子库中，从而把证明检索转化为“每色一次 PIR”的批量检索过程。然而，原始 TreePIR 的颜色可行性、分裂算法和索引方法都建立在 perfect tree 的高度规则结构之上。当数据本身是一棵固定但不规则的 Merkle 树时，直接将其改写为 perfect tree 再处理，往往会引入额外结构转换，并削弱方法对原始数据组织的适配性。

本文的核心出发点是：保留 swapped tree 的逻辑意义，但不再要求显式构造一棵完整的 swapped perfect tree。更具体地，我们只保留那些会在至少一个真实证明中出现的节点，将其称为活跃证明节点（active proof-bearing nodes），并在这些节点上直接进行祖先染色。这样做有三个直接好处。第一，证明在查询视角下仍然是一条逻辑路径，因此 TreePIR 的祖先染色思想仍然成立。第二，默认空节点不进入子数据库，总存储只与真实数据相关。第三，利用节点对应的真实叶区间，可以直接构建子库索引，而无需再下载接近整棵树规模的额外映射。

本文的主要贡献如下：

1. 提出 virtual-swapped sparse view, 用于将任意固定二叉 Merkle 树上的真实证明节点组织为适合 batch-PIR 的逻辑查询对象。
2. 给出一种面向任意固定树的直接祖先染色框架, 并证明只要路径异色约束成立, 客户端即可通过一次包含 h 个 PIR 子查询的 batch-PIR 请求恢复完整证明。
3. 设计基于叶区间的子库索引方法, 并证明同色区间不相交, 因此单次查询仅需对每个颜色子库执行一次二分搜索即可确定真实索引或 dummy。
4. 实现两种不同的染色目标函数, 并在随机生成的不规则 Merkle 树上比较子库规模平衡与加权负载平衡之间的权衡关系。

2 模型与问题定义

2.1 Virtual-swapped sparse view

设 T 是一棵固定高度为 h 的二叉 Sparse Merkle Tree, 真实叶集合记为 $\mathcal{L}$ 。Merkle tree 的哈希构造仍然按通常方式自底向上进行; 本文中的 swapped 仅用于 proof 的逻辑组织与查询建模, 而不改变哈希计算过程。

定义 2.1 (活跃证明节点). 设 u 是原树中的一个非根节点, $\text{sib}(u)$ 为其兄弟节点。若 $\text{sib}(u)$ 的子树中包含至少一个真实叶, 则称 u 为一个活跃证明节点。记所有活跃证明节点组成的集合为 $\mathcal{A}(T)$ 。

定义的含义十分直接: 节点 u 会出现在某个真实叶的 membership proof 中, 当且仅当其兄弟子树中至少存在一个真实叶。由于默认空子树的哈希值可由客户端公开重建, 因此不属于 $\mathcal{A}(T)$ 的节点无需进入 PIR 子数据库。

定义 2.2 (Virtual-swapped proof path). 对任意真实叶 $\ell \in \mathcal{L}$, 记

$$\text{Path}(\ell) \subseteq \mathcal{A}(T)$$

为叶 ℓ 的完整 Merkle proof 中所有真实且需要私有检索的节点, 按照 swapped 视角下从祖先到后代的顺序排列得到的节点序列, 称其为叶 ℓ 的活跃 swapped proof path。

虽然固定高度 SMT 的完整证明长度始终为 h , 但活跃 swapped proof path 中的真实节点数量可以小于 h , 因为某些层只对应公开可重建的默认空节点。

2.2 叶区间表示

为了构建高效索引, 我们对每个活跃节点引入叶区间表示。将真实叶按从左到右编号。对于任意活跃节点 $u \in \mathcal{A}(T)$, 定义

$$I(u) = [L(u), R(u)]$$

为其兄弟子树在真实叶序上的覆盖区间。于是, 对于任意真实叶 ℓ ,

$$u \in \text{Path}(\ell) \iff \ell \in I(u).$$

由于二叉树中任一子树都对应一段连续叶区间, 因此这些区间构成一个层叠 (laminar) 区间族: 任意两个区间要么不相交, 要么一个包含另一个。这一结构将成为后续染色与索引的共同基础。

2.3 祖先染色与优化目标

本文统一使用 h 种颜色，对活跃证明节点进行着色。

定义 2.3 (合法祖先染色). 一个映射

$$\varphi : \mathcal{A}(T) \rightarrow [h]$$

称为一个合法祖先染色，若对任意真实叶 $\ell \in \mathcal{L}$ ，路径 $\text{Path}(\ell)$ 上任意两个不同节点 u, v 满足

$$u, v \in \text{Path}(\ell), u \neq v \implies \varphi(u) \neq \varphi(v).$$

合法祖先染色保证同一条真实证明路径上的节点颜色互异，因此客户端在每个颜色子库中至多需要检索一个真实节点。

本文考虑以下两类平衡目标。设第 c 个颜色子库为

$$\mathcal{D}_c = \{u \in \mathcal{A}(T) : \varphi(u) = c\},$$

其大小记为 $N_c = |\mathcal{D}_c|$ 。若将节点 u 的权重定义为其区间长度

$$w(u) = |I(u)|,$$

则第 c 个颜色子库的加权负载为

$$W_c = \sum_{u: \varphi(u)=c} w(u).$$

本文实现两种启发式优化目标：

1. **weighted**: 优先减小 $\max_c W_c - \min_c W_c$;
2. **count_balanced**: 优先减小 $\max_c N_c - \min_c N_c$ 。

3 直接祖先染色算法

3.1 活跃节点与区间森林的构造

我们的实现不显式生成完整 swapped tree，而是直接从原始 Merkle tree 生成活跃节点及其区间表示，再根据区间包含关系构造一棵“冲突森林”。其核心思想是：若两个活跃节点会在某条证明中共同出现，则对应区间必然相交；而在 laminar 家族中，相交进一步等价于包含关系。

算法 1 活跃节点与区间森林构造

输入: 固定二叉 Merkle tree T , 真实叶集合 $\mathcal{L}$ **输出:** 活跃节点集合 $\mathcal{A}(T)$, 区间森林 $\mathcal{F}$

- 1: 对 T 进行一次 DFS, 计算每个节点对应的真实叶区间和真实叶数
 - 2: 初始化 $\mathcal{A}(T) \leftarrow \emptyset$
 - 3: **for** 每个非根节点 u **do**
 - 4: $s \leftarrow \text{sib}(u)$
 - 5: **if** s 的子树中包含至少一个真实叶 **then**
 - 6: 将 u 加入 $\mathcal{A}(T)$
 - 7: 为 u 赋予区间 $I(u) = [L(s), R(s)]$ 与权重 $w(u) = |I(u)|$
 - 8: **end if**
 - 9: **end for**
 - 10: 将所有区间按左端点升序、区间长度降序排序
 - 11: 使用栈按区间包含关系建立森林 $\mathcal{F}$
 - 12: **return** $\mathcal{A}(T), \mathcal{F}$
-

3.2 目标驱动的祖先染色

在得到区间森林后, 我们采用“贪心着色 + 局部重平衡”的两阶段方法。贪心阶段自上而下遍历区间森林, 在当前节点可用的颜色中, 根据目标函数选择最优颜色; 局部重平衡阶段则尝试将节点从较重颜色搬移到较轻颜色, 只要不破坏祖先异色约束且目标值下降就接受。

算法 2 目标驱动的直接祖先染色**输入:** 区间森林 $\mathcal{F}$, 颜色数 h , 目标类型 $\text{OBJ} \in \{\text{weighted}, \text{count_balanced}\}$, 最大重平衡轮数 R **输出:** 合法祖先染色 φ

```

1: 初始化所有颜色的节点数负载  $\{N_c\}_{c=1}^h$  与加权负载  $\{W_c\}_{c=1}^h$  为 0
2: if  $\text{OBJ} = \text{count\_balanced}$  then
3:   根据  $|\mathcal{A}(T)|$  为每个颜色设置目标配额  $q_c \in \{\lfloor |\mathcal{A}(T)|/h \rfloor, \lceil |\mathcal{A}(T)|/h \rceil\}$ 
4: end if
5: 按权重降序排列森林根节点
6: for 每棵根节点为  $r$  的树 do
7:   从  $r$  开始执行深度优先着色
8:   for 访问到的每个节点  $u$  do
9:     令  $\mathcal{C}(u)$  为所有未出现在祖先路径中的可用颜色
10:    按目标函数在  $\mathcal{C}(u)$  中选取最优颜色  $c^*$ 
11:    赋值  $\varphi(u) \leftarrow c^*$ , 并更新对应的  $N_{c^*}, W_{c^*}$ 
12:   end for
13: end for
14: for  $t = 1$  to  $R$  do
15:   在所有合法重染色移动中寻找一个能改进目标函数的最佳移动
16:   if 不存在可改进移动 then
17:     break
18:   else
19:     执行该重染色并更新负载
20:   end if
21: end for
22: return  $\varphi$ 

```

算法 2 中的目标函数具体取值如下。

Weighted 目标. 贪心着色时, 优先选择使

$$(W_c, N_c, |N_c - q_c|, c)$$

按字典序最小的颜色; 局部重平衡时, 优先最小化

$$(\max_c W_c - \min_c W_c, \max_c N_c - \min_c N_c, \max_c W_c, \max_c N_c).$$

Count-balanced 目标. 贪心着色时, 优先选择使

$$(N_c - q_c, N_c, W_c, c)$$

按字典序最小的颜色; 局部重平衡时, 优先最小化

$$(\max_c N_c - \min_c N_c, \max_c N_c, \max_c W_c - \min_c W_c, \max_c W_c).$$

注记 3.1. 上述染色算法是一个面向任意固定 *Merkle tree* 的直接启发式, 而非全局最优算法。本文关注的是: 在保持祖先异色约束与 *batch-PIR* 兼容性的前提下, 比较不同平衡目标对颜色子库划分的影响。

3.3 基于区间索引的 batch-PIR 查询生成

着色完成后, 客户端需要在每个颜色子库中找到目标叶对应的真实节点或 dummy 位置。由于同色区间不相交, 客户端只需在每个颜色子库的有序区间表中执行一次二分即可。

算法 3 基于区间索引的 batch-PIR 查询生成

输入: 目标叶 ℓ , 颜色子库 $\{\mathcal{D}_c\}_{c=1}^h$, 同色有序区间表 $\{\mathcal{I}_c\}_{c=1}^h$

输出: 一个包含 h 个 PIR 子查询的 batch-PIR 请求

```

1: for  $c = 1$  to  $h$  do
2:   在  $\mathcal{I}_c$  中对叶  $\ell$  执行二分搜索
3:   if 存在唯一一个区间包含  $\ell$  then
4:     令该节点在  $\mathcal{D}_c$  中的位置为  $j_c$ 
5:     生成面向  $\mathcal{D}_c[j_c]$  的真实 PIR 子查询
6:   else
7:     生成面向  $\mathcal{D}_c$  的 dummy PIR 子查询
8:   end if
9: end for
10: 将全部  $h$  个 PIR 子查询同时发送到对应颜色子库
11: 接收并合并全部响应, 恢复目标叶的真实证明节点

```

4 正确性与复杂度分析

命题 4.1 (唯一性). 若 φ 是一个合法祖先染色, 则对于任意真实叶 $\ell \in \mathcal{L}$ 和任意颜色 $c \in [h]$, 路径 $\text{Path}(\ell)$ 上至多存在一个颜色为 c 的活跃节点。

证明. 若存在两个不同节点 $u, v \in \text{Path}(\ell)$ 满足 $\varphi(u) = \varphi(v) = c$, 则与合法祖先染色定义矛盾。□

命题 4.2 (batch-PIR 兼容性). 设底层单项 PIR 方案是正确的。若 φ 是一个合法祖先染色, 则客户端通过一次包含 h 个 PIR 子查询的 batch-PIR 请求, 可以恢复目标叶 ℓ 的全部真实证明节点。

证明. 由命题 4.1 可知, 对任意颜色 c , 路径 $\text{Path}(\ell)$ 中至多存在一个颜色为 c 的真实节点。因此, 在每个颜色子库中客户端至多检索一个真实条目; 若该颜色上不存在真实节点, 则发送 dummy 查询即可。收集所有子查询返回的真实结果, 并与公开可重建的默认空哈希组合, 即可恢复完整 proof。□

命题 4.3 (隐私兼容性). 若底层单项 PIR 满足计算隐私性, 则上述 batch-PIR 检索方案对目标叶索引保持计算隐私。

证明. 客户端在每次检索时始终同时向全部 h 个颜色子库发送 PIR 子查询。对某些颜色, 这些子查询对应真实节点; 对另一些颜色, 它们对应 dummy。由于底层 PIR 已保证单个子查询内容不可区分, 服务器观察到的始终是一个固定结构的 batch-PIR 请求, 因此无法判断目标叶对应的真实证明路径。□

引理 4.1 (同色区间不相交). 若 φ 是一个合法祖先染色, 则对于任意颜色 c , 集合

$$\mathcal{I}_c = \{I(u) : \varphi(u) = c\}$$

中的任意两个不同区间必不相交。

证明. 设存在两个不同节点 u, v 满足 $\varphi(u) = \varphi(v) = c$ 且 $I(u) \cap I(v) \neq \emptyset$. 取任意真实叶 $\ell \in I(u) \cap I(v)$, 则根据区间定义, u 与 v 同时出现在 ℓ 的证明中, 即 $u, v \in \text{Path}(\ell)$. 这与路径异色约束矛盾。□

定理 4.1 (索引复杂度). 设 $N = |\mathcal{A}(T)|$ 为活跃证明节点总数, $N_c = |\mathcal{D}_c|$ 为颜色 c 的子库大小. 则基于叶区间的预处理和查询复杂度分别满足:

预处理时间 $\in O(N \log N)$,

预处理空间 $\in O(N)$,

单次 *batch-PIR* 查询索引时间 $\in O(h \log N)$.

证明. 首先, 遍历原树并计算所有活跃节点区间的时间为 $O(N)$. 然后, 对每个颜色类的区间表进行排序, 总时间为

$$O\left(\sum_{c=1}^h N_c \log N_c\right) \subseteq O(N \log N).$$

排序后, 客户端对每个颜色子库做一次二分搜索, 因此单次查询的索引时间为

$$O\left(\sum_{c=1}^h \log N_c\right) \subseteq O(h \log N).$$

总空间显然为存储所有活跃节点及其区间表所需的 $O(N)$ 。□

5 实验评估

5.1 实验设置

为了评估不同染色目标对 virtual-swapped TreePIR 子库划分效果的影响, 我们比较了两种策略: 以加权负载平衡为主目标的 **weighted** 策略, 以及以子库大小平衡为主目标的 **count_balanced** 策略。

实验对象为随机生成的不规则二叉 Merkle tree。具体地, 我们从给定数量的叶节点出发, 保持叶的左右顺序不变, 并通过“随机选择一对相邻子树进行合并”的方式自底向上构造一棵随机树。该构造既符合 Merkle tree 的生成方式, 又能产生丰富的非平衡树形。对每棵树, 我们在相同结构上分别运行两种策略, 从而进行公平比较。

叶节点数量分别设置为 8, 12, 16, 20。对每一组规模, 生成 100 棵随机树, 因此总实验次数为 400。局部重平衡轮数统一设置为 400。对每棵树, 我们记录如下三个指标:

1. 子库大小差值 $\max_c N_c - \min_c N_c$;
2. 加权负载差值 $\max_c W_c - \min_c W_c$;
3. 祖先性质通过率, 即染色是否满足所有真实 swapped proof path 上颜色互异。

5.2 实验结果

表 1 给出了两种策略在不同叶规模下的平均结果。所有数值均为对应 100 棵随机树的平均值。

表 1: 两种染色策略在随机 Merkle tree 上的对比

叶节点数	weighted			count_balanced		
	平均子库大小差值	平均加权负载差值	通过率	平均子库大小差值	平均加权负载差值	通过率
8	3.250	1.200	100.0%	1.880	4.000	100.0%
12	4.500	1.920	100.0%	2.870	6.680	100.0%
16	5.810	2.540	100.0%	3.600	9.580	100.0%
20	7.370	3.660	100.0%	4.510	12.500	100.0%

进一步将全部 400 次实验合并统计，可得：

- **weighted**: 平均子库大小差值为 5.232, 平均加权负载差值为 2.330, 祖先性质通过率为 100.0%;
- **count_balanced**: 平均子库大小差值为 3.215, 平均加权负载差值为 8.190, 祖先性质通过率为 100.0%。

5.3 结果分析

实验结果表明，两种策略在优化目标上呈现出清晰的权衡关系。

首先，**count_balanced** 在所有叶规模下都显著降低了颜色子库的大小差值。例如在叶节点数为 20 时，平均子库大小差值从 7.370 降低到 4.510；在总体统计中，该指标从 5.232 降低到 3.215。这说明若系统的主要目标是减小不同颜色子库之间的存储规模差异，则以子库大小平衡为主目标的策略更合适。

其次，这种改进是以牺牲加权负载平衡为代价的。总体上，**weighted** 的平均加权负载差值仅为 2.330，而 **count_balanced** 上升到 8.190。这说明当策略优先追求“每个颜色子库含有相近数量的节点”时，某些高权重节点会更集中地分配到少数颜色中，从而导致不同颜色上的查询压力更加不均衡。

再次，祖先性质在全部实验中均保持 100.0% 通过率。这表明无论优化目标是子库大小还是加权负载，当前两种染色算法都能稳定满足 virtual-swapped TreePIR 所要求的路径异色约束。因此，二者的差异主要体现为平衡目标不同，而非正确性不同。

6 结论与后续工作

本文提出了一种面向任意固定 Merkle tree 的 virtual-swapped TreePIR 框架。该框架保留了 TreePIR 中 swapped-path 与祖先染色的核心思想，但不再要求输入必须是 perfect tree，也不需要默认空节点写入子数据库。通过对真实活跃证明节点的直接着色与基于叶区间的索引，客户端可以以一次 batch-PIR 请求的形式恢复目标证明，并在理论上保持正确性、隐私兼容性与对数级索引开销。

随机树实验说明，在任意固定 Merkle tree 的祖先染色问题中，子库规模均衡与查询负载均衡并不天然一致。**count_balanced** 更适合优化存储规模均衡，而 **weighted** 更适合优化查询负载均衡。因此，一个自然的后续问题是：如何设计同时兼顾子库大小与加权负载的多目标折中染色策略，并进一步分析其在实际 PIR 开销模型下的端到端收益。

参考文献

- [1] S. H. Dau, Q. Cao, R. Gagliano, D. Huynh, X. Yi, P. L. Le, Q.-H. Luu, E. Viterbo, Y.-C. Huang, J. Zhu, M. M. Jalalzai, and C. Feng, “TreePIR: Efficient Private Retrieval of Merkle Proofs via Tree Colorings with Fast Indexing and Zero Storage Overhead,” arXiv preprint arXiv:2205.05211v5, 2024.
- [2] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, “Private information retrieval,” *Journal of the ACM*, vol. 45, no. 6, pp. 965–981, 1998.